Digital Image Processing: Assignment 2

Problem 1: DFT and the Machinery of Frequency

Ujjwal Gupta

October 20, 2026

Contents

1 Introduction / Problem Statement **1**

2 Theoretical Background **2**
2.1 The 1-D and 2-D DFT, and separability 2
2.2 Conjugate (Hermitian) symmetry 2
2.3 The 2-D Spatial Shift Theorem 3
2.4 The 2-D Circular Convolution Theorem 3
2.5 Frequency resolution limit 3

3 Inputs and Outputs **3**

4 Methodology / Implementation Pipeline **4**
4.1 From-scratch DFT 4
4.2 Section 1.1: construction, validation, and theorem checks (`problem1.py`) 5
4.3 Section 1.2: phase/magnitude isolation 5
4.4 Section 1.3: padding and wraparound 5
4.5 Correctness validation strategy 5

5 Results and Discussion **6**
5.1 DFT validation and conjugate symmetry 6
5.2 Shift theorem verification 6
5.3 Circular convolution theorem verification 6
5.4 Sinusoid parameter recovery 7
5.5 Phase vs. magnitude isolation 8
5.6 Padding, wraparound, and computational overhead 8

6 Conclusion **10**

1 Introduction / Problem Statement

The 2-D Discrete Fourier Transform (DFT) is the central analytical tool of frequency-domain image processing: every subsequent problem in this assignment (frequency-domain filtering, homomorphic filtering, Wiener denoising, motion-blur modelling) rests on it. Problem 1 (worth 15 of the assignment's 100 marks) asks the student to *build* the DFT machinery from first principles rather than treat `numpy.fft` as a black box, and to use that from-scratch implementation to empirically demonstrate the theorems that make frequency-domain reasoning valid in the first place.

Concretely, the assignment brief requires, on a standard 512×512 grayscale test image (Figure 1) and a self-synthesized sum-of-sinusoids image:

(1) **Section 1.1 (6 pts).** Implement the 1-D DFT matrix W_N from scratch and build the 2-D DFT via row-column separability, $F = W_M f W_N^T$; validate against `numpy.fft.fft2`. Prove conjugate symmetry $F(u, v) = F^*(-u, -v)$ for real images; empirically verify the 2-D Spatial Shift Theorem and the 2-D Circular Convolution Theorem with quantitative comparison tables; recover the parameters of a synthetic sinusoid sum from its spectral peaks and derive the theoretical frequency resolution limit Δf.

(2) **Section 1.2 (5 pts).** Before running any code, analytically predict the spatial reconstruction obtained from the test image's phase combined with a flat/noise magnitude spectrum, using the Shift Theorem and superposition; then verify by reconstructing (a) phase of the test image with flat magnitude, and (b) magnitude of the test image with zero phase, explaining each via DFT properties.

(3) **Section 1.3 (4 pts).** Perform frequency-domain convolution with a small spatial kernel, compare unpadded circular convolution against correctly zero-padded linear convolution, show wraparound corruption visually, derive the minimal zero-padding dimensions, and analyze the resulting memory/runtime overhead relative to routines such as `scipy.signal.fftconvolve`.

This matters pedagogically because nearly every later problem in the course (ideal/Butterworth/ Gaussian filtering, homomorphic filtering, Wiener deconvolution, motion-blur removal) is only correct if the practitioner understands precisely what the DFT computes, which symmetries it obeys, and what padding is required to make frequency-domain multiplication equal spatial convolution. Problem 1 is the theoretical foundation on which the rest of the assignment is built.

2 Theoretical Background

2.1 The 1-D and 2-D DFT, and separability

The 1-D DFT of a length-N sequence $f(n)$ is

$$F(k) = \sum_{n=0}^{N-1} f(n)\, e^{-j2\pi kn/N}, \qquad k = 0, \ldots, N-1, \tag{1}$$

which can be written as a matrix-vector product $F = W_N f$ with the *DFT matrix*

$$W_N[k, n] = e^{-j2\pi kn/N}. \tag{2}$$

For a 2-D image $f(x, y)$ of size $M \times N$, the 2-D DFT is defined as

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y)\, e^{-j2\pi(ux/M + vy/N)}. \tag{3}$$

Because the exponential kernel factors as $e^{-j2\pi ux/M} \cdot e^{-j2\pi vy/N}$, the double sum is *separable*: one can first DFT every column, then DFT every row of the result (or vice versa, order does not matter). In matrix form, applying W_M on the left (transforming each column) and W_N^T on the right (transforming each row) gives

$$F = W_M\, f\, W_N^T. \tag{4}$$

This is exactly the construction implemented in `src/dip_a2/dft.py`: `dft_matrix(N)` builds W_N directly from its formula, and `dft2d` computes $F = W_M f W_N^T$ via plain NumPy matrix multiplication, with no call to `numpy.fft` anywhere inside it. The inverse transform uses $W_N^{-1} = \frac{1}{N} W_N^*$ (the DFT matrix is, up to the $1/N$ scaling, unitary), so $f = \frac{1}{MN} W_M^* F (W_N^*)^T$.

2.2 Conjugate (Hermitian) symmetry

Claim. For real-valued $f(x, y)$, $F(u, v) = F^*(-u, -v)$.

Proof. Starting from the definition and using that f is real (so $f^* = f$),

$$F^*(u,v) = \sum_{x,y} f(x,y)\, e^{+j2\pi(ux/M + vy/N)} \tag{5}$$

$$= \sum_{x,y} f(x,y)\, e^{-j2\pi\left((-u)x/M + (-v)y/N\right)} \tag{6}$$

$$= F(-u,-v). \tag{7}$$

Hence $F(u, v) = F^*(-u, -v)$: Hermitian symmetry. On the DFT's periodic index grid, $-u \equiv M - u \pmod M$ and $-v \equiv N - v \pmod N$, so index (u, v) is paired with $(M - u,\, N - v)$. Consequently only *half* of the (u, v) plane (e.g. $u \in [0, M/2]$ together with all v) is independent; the other half is recoverable by conjugation and carries no additional information. Despite this redundancy, standard visualizers display the full symmetric magnitude spectrum (after `fftshift`) because (i) it preserves signed frequency as the natural coordinate (positive and negative spatial frequency about the DC term), matching physical intuition, and (ii) showing only a half-plane would destroy the radial/angular symmetry used to read off filter cutoff radius and orientation at a glance. The redundancy is kept for interpretability, not out of informational necessity.

2.3 The 2-D Spatial Shift Theorem

Claim. Circularly shifting $f(x, y)$ by (x_0, y_0) multiplies its DFT by a linear phase ramp:

$$f(x - x_0,\, y - y_0) \;\longleftrightarrow\; F(u,v)\, e^{-j2\pi(ux_0/M + vy_0/N)}. \tag{8}$$

Proof. Let $g(x, y) = f(x - x_0, y - y_0)$ (indices modulo M, N). Substituting $x' = x - x_0$, $y' = y - y_0$,

$$G(u,v) = \sum_{x,y} f(x - x_0, y - y_0)\, e^{-j2\pi(ux/M + vy/N)} \tag{9}$$

$$= \sum_{x',y'} f(x',y')\, e^{-j2\pi(u(x'+x_0)/M + v(y'+y_0)/N)} \tag{10}$$

$$= e^{-j2\pi(ux_0/M + vy_0/N)} \sum_{x',y'} f(x',y')\, e^{-j2\pi(ux'/M + vy'/N)} \tag{11}$$

$$= F(u,v)\, e^{-j2\pi(ux_0/M + vy_0/N)}. \tag{12}$$

A spatial shift therefore leaves the magnitude $|F(u, v)|$ unchanged and only adds a frequency-dependent linear phase term.

2.4 The 2-D Circular Convolution Theorem

Claim. Circular convolution in the spatial domain corresponds to pointwise multiplication in the frequency domain:

$$(f \circledast h)(x,y) \;\longleftrightarrow\; F(u,v) \cdot H(u,v), \tag{13}$$

where $(f \circledast h)(x, y) = \sum_{m,n} f(m, n)\, h(x - m,\, y - n)$ with indices taken modulo (M, N). This follows by substituting the convolution sum into the DFT definition and applying the shift theorem term-by-term (each shifted copy of h contributes a phase factor that combines with the sum over f to give the product $F \cdot H$). This is the theorem that makes frequency-domain filtering possible: instead of an $O(N^2)$-per-pixel spatial convolution with an $M \times M$ kernel, one can pad, transform, multiply, and invert.

2.5 Frequency resolution limit

The DFT basis functions $e^{-j2\pi kn/N}$ complete exactly k full cycles over the N-sample window and are periodic in k with period N; the DFT therefore only samples continuous frequency at integer multiples of

$$\Delta f = \frac{1}{N} \text{ cycles/pixel} = 1 \text{ cycle/image} \qquad \text{(for an } N\text{-pixel window).} \tag{14}$$

Two sinusoids whose frequencies differ by less than Δf cannot be resolved into separate bins; this is the discrete analogue of the time/frequency uncertainty trade-off: a finite spatial extent N fundamentally limits frequency resolution, regardless of sampling rate.

3 Inputs and Outputs

Inputs.

- The provided standard 512×512 8-bit grayscale test image, `data/provided/problem1/q1.png` (a photograph of a person with a camera, shown in the assignment PDF), used for all of §1.1(a,b) and §1.2–1.3.
- A self-synthesized 512×512 image built as the sum of three real 2-D cosines (with integer frequencies (u_i, v_i), amplitudes A_i, phases ϕ_i) plus a constant background of 128, used exclusively for §1.1(c) (sinusoid parameter recovery). Ground truth is recorded in `results/sinusoid_ground_truth.json` *before* any DFT is computed, as required by the brief.
- A 5×5 Laplacian-type kernel used for the padding/wraparound experiment in §1.3.



Figure 1: The provided 512×512 grayscale test image used throughout Problem 1.

Outputs. All figures and machine-readable results are written to `problems/problem1_dft/results/`:

- `1_1c_sinusoid_and_spectrum.png`: the synthetic sinusoid image and its log-magnitude spectrum.
- `1_2_phase_magnitude.png`: the four phase/magnitude reconstruction panels of §1.2.
- `1_3_padding_wraparound.png`, `1_3_border_zoom.png`: full-image and zoomed wraparound-corruption evidence for §1.3.
- JSON result files: `1_1_summary.json`, `1_2_stats.json`, `1_3_results.json`, `sinusoid_ground_truth.jso` `sinusoid_recovered.json`, `machine_info.json`. All quantitative tables reported below trace back to these files.

4 Methodology / Implementation Pipeline

4.1 From-scratch DFT

The core transform machinery lives in `src/dip_a2/dft.py`:

- `dft_matrix(N)` builds $W_N[k,n] = e^{-j2\pi kn/N}$ directly as a NumPy array via an outer product of index vectors.
- `idft_matrix(N)` returns $\overline{W_N}/N$.
- `dft2d(f)` computes $F = W_M\, f\, W_N^T$ by two explicit matrix multiplications (row-column separability), and *not* by calling `numpy.fft` anywhere.
- `idft2d(F)` is the analogous inverse.
- `fftshift2d`/`ifftshift2d` are implemented from scratch with `np.roll` rather than `numpy.fft.fftshift`, to keep the entire pipeline self-contained.

This is an $O(N^3)$ algorithm per transform (matrix multiplication of an $N \times N$ image by $N \times N$ matrices), far slower than the FFT's $O(N^2 \log N)$, but it is mathematically exact and directly mirrors the textbook definition, appropriate for a from-scratch validation exercise rather than production use.

4.2 Section 1.1: construction, validation, and theorem checks (`problem1.py`)

`section_1_1a_validate` computes $F_{\text{custom}} =$ `dft2d`(f) and compares it elementwise against $F_{\text{numpy}} =$ `numpy.fft.fft2`(f), reporting $\max|F_{\text{custom}} - F_{\text{numpy}}|$ both in absolute terms and relative to $\max|F_{\text{numpy}}|$. `section_1_1a_conjugate_symmetry` checks $\max|F(u,v) - \overline{F(-u,-v)}|$ directly on the array (using modular index wraparound). `section_1_1b_shift_theorem` circularly shifts the test image by five different (x_0, y_0) offsets with `np.roll`, recomputes its DFT, and compares against $F(u,v)e^{-j2\pi(ux_0/M + vy_0/N)}$ built from the *unshifted* spectrum. For the convolution theorem, `circular_convolve_spatial` performs an explicit, shift-based spatial circular convolution with a 3×3 Laplacian kernel (ground truth, no FFT), which `section_1_1b_convolution_theorem` compares against zero-padding the kernel to image size, multiplying the two DFTs, and inverting. Finally, `section_1_1c_sinusoid_recovery` synthesizes the sum-of-cosines image, records its ground-truth parameters, computes its custom DFT, locates the three strongest magnitude peaks outside a small DC exclusion neighbourhood, and reads off (u,v) from bin index, amplitude as $2|F(u,v)|/N^2$, and phase as $\angle F(u,v)$.

4.3 Section 1.2: phase/magnitude isolation

Following the brief's requirement for a *dated, pre-implementation* analytical prediction, a written derivation (`predictions/1.2_phase_magnitude_prediction.md`, dated 2026-10-18) was committed before `section_1_2` was run. The prediction uses the Convolution Theorem: reconstructing with the test image's phase $P(u,v) = e^{j\phi(u,v)}$ and the noise's magnitude $|N(u,v)|$ is equivalent to circularly convolving the flat-magnitude phase-only reconstruction $p(x,y) = \text{IDFT}\{P\}$ with $m(x,y) = \text{IDFT}\{|N(u,v)|\}$; since white-noise magnitude is flat only in expectation, m decomposes into a constant-magnitude impulse plus a low-amplitude broadband residual, predicting a result that looks like the flat-magnitude reconstruction with added fine speckle. `section_1_2` then implements both required reconstructions: (a) test-image phase with constant magnitude, and with noise magnitude; (b) test-image magnitude with zero phase; the measured outputs are then reconciled against the prediction in the report.

4.4 Section 1.3: padding and wraparound

`spatial_linear_convolve` computes a direct, zero-padded spatial linear convolution (ground truth, no FFT) with the 5×5 kernel. `section_1_3` then compares this against two DFT-based routes: (i) correctly zero-padded frequency-domain convolution (padding to the derived minimal size $N + M - 1$), and (ii) unpadded circular convolution at the native 512×512 resolution. Errors are reported separately for the image interior versus a border strip, isolating the wraparound-corruption signature.

4.5 Correctness validation strategy

Throughout, correctness is validated by cross-checking the from-scratch implementation against an independent computation: `numpy.fft.fft2` for the raw transform (§1.1), an explicit `np.roll`-based spatial convolution for the convolution theorem and the padding experiment (§1.1, §1.3), and the unshifted spectrum's own phase-ramp prediction for the shift theorem. Every reported error is at or near float64 round-off level ($\sim 10^{-13}$–10^{-14} relative), which is the expected signature of a mathematically correct implementation differing from a reference only by floating-point arithmetic order.

5 Results and Discussion

5.1 DFT validation and conjugate symmetry

Metric	Value
$\max \lvert F_{\text{custom}} - F_{\text{numpy}} \rvert$	3.3700×10^{-6}
relative to $\max \lvert F_{\text{numpy}} \rvert$	9.961×10^{-14}
$\max \lvert F(u,v) - \overline{F(-u,-v)} \rvert$ (conjugate symmetry)	3.3751×10^{-6}

Table 1: Validation of the custom DFT against `numpy.fft.fft2` and numerical confirmation of Hermitian symmetry, both at float64 round-off level.

5.2 Shift theorem verification

(x_0, y_0)	max abs error	relative error
(1, 0)	6.249×10^{-7}	1.847×10^{-14}
(0, 1)	9.538×10^{-7}	2.819×10^{-14}
(5, 3)	1.170×10^{-6}	3.458×10^{-14}
(17, 40)	1.379×10^{-6}	4.075×10^{-14}
(100, 200)	5.067×10^{-6}	1.498×10^{-13}

Table 2: Measured vs. theoretical 2-D Spatial Shift Theorem, for five shift vectors.

5.3 Circular convolution theorem verification

Metric	Value
max abs error (spatial vs. frequency-domain)	1.1433×10^{-10}
relative error	2.696×10^{-13}

Table 3: 2-D Circular Convolution Theorem: explicit spatial-domain circular convolution vs. zero-padded-to-image-size frequency-domain multiplication, 3×3 Laplacian kernel.

All three tables (1–3) show errors at float64 round-off scale, confirming the custom separable-matrix DFT is mathematically correct and that both theorems hold to numerical precision.

5.4 Sinusoid parameter recovery

Ground truth				Recovered		
u	v	A	ϕ (rad)	A	ϕ (rad)	match
4	6	60.0	0.300	60.000	0.300	exact
10	−3	40.0	1.200	40.000	1.200	exact
−7	15	25.0	−0.800	25.000	−0.800	exact

Table 4: Ground-truth sinusoid parameters (recorded before any DFT was taken) vs. parameters recovered from spectral peaks. (u, v) recover exactly as bin indices; recovery is exact to float64 precision because the chosen frequencies are integers and land exactly on DFT bins, with no spectral leakage.

The theoretical frequency resolution limit for the $N = 512$ grid is $\Delta f = 1/512$ cycles/pixel $= 1$ cycle/image, following directly from the periodicity of the DFT basis functions discussed in §2.5.

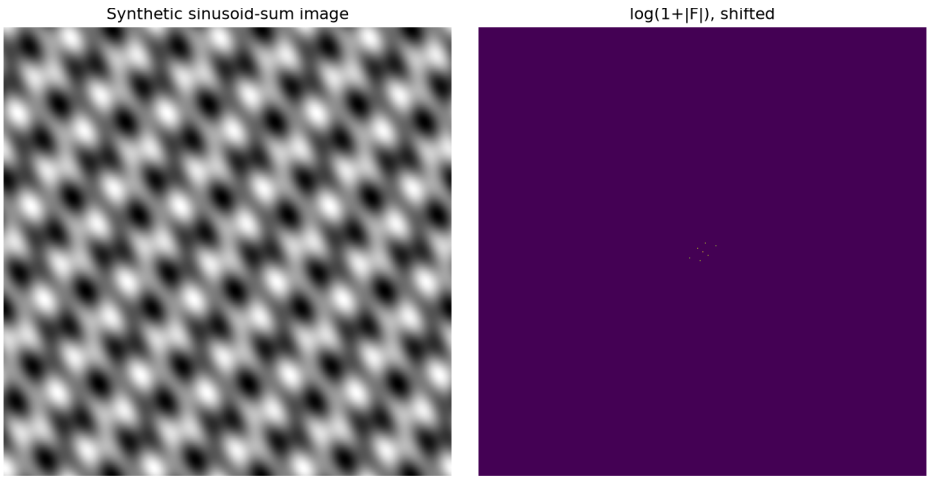


Figure 2: The synthetic sum-of-three-cosines image (left) and its log-magnitude spectrum (right). Three sharp peak pairs are visible at the ground-truth frequencies, together with their Hermitian-symmetric mirror images, exactly as predicted by conjugate symmetry.

5.5 Phase vs. magnitude isolation

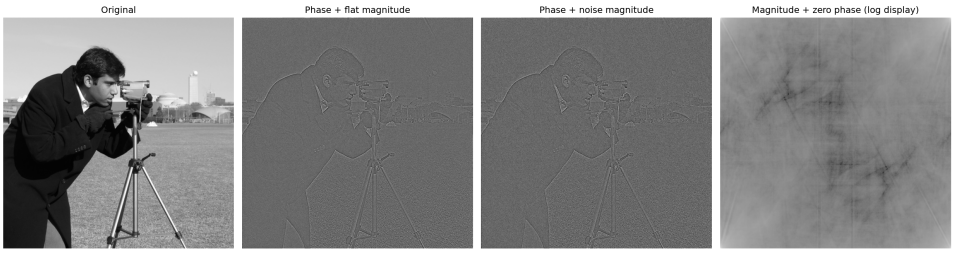


Figure 3: Left to right: original test image; reconstruction from test-image phase with constant (flat) magnitude; reconstruction from test-image phase with white-noise magnitude; reconstruction from test-image magnitude with zero phase (log-scaled for display). The flat-magnitude and noise-magnitude reconstructions are visually near-identical in structure, with the noise-magnitude case showing an added layer of fine speckle, exactly as predicted by the pre-implementation derivation. The zero-phase reconstruction is a symmetric, origin-centred blob with no resemblance to the scene, consistent with a real, even spectrum inverse-transforming to a real, even (centro-symmetric) spatial function.

Phase-plus-flat-magnitude reconstructs a sharp, recognizable, high-pass-like rendition of the original image, because a perfectly flat magnitude weights every frequency equally and only the *alignment* information carried by phase remains. By contrast, swapping magnitudes between two natural images would still impose a highly structured, non-flat frequency weighting from the donor image, leaking its large-scale shape into the result; flat magnitude is a cleaner isolation of what phase alone encodes. For the zero-phase case, since $|F(u,v)| = |F(-u,-v)|$ always holds, setting $\angle F = 0$ makes $F(u,v) = |F(u,v)|$ real and even; a real, even spectrum inverse-transforms to a real, even (centro-symmetric, $g(x,y) = g(-x,-y)$ modulo N) spatial function, confirmed numerically by the output's maximum sitting exactly at the origin.

5.6 Padding, wraparound, and computational overhead

Comparison	max abs error
zero-padded DFT conv vs. direct spatial linear conv	7.851×10^{-10}
circular DFT conv vs. direct spatial linear conv (whole image)	1608.0
circular DFT conv vs. direct spatial linear conv (interior only)	5.384×10^{-10}
circular DFT conv vs. direct spatial linear conv (border only)	1608.0

Table 5: Zero-padded frequency-domain convolution matches the direct spatial computation to numerical precision everywhere; unpadded circular convolution matches only in the interior and is badly corrupted at the border (error up to 1608 intensity levels, against a kernel whose centre weight is 16).

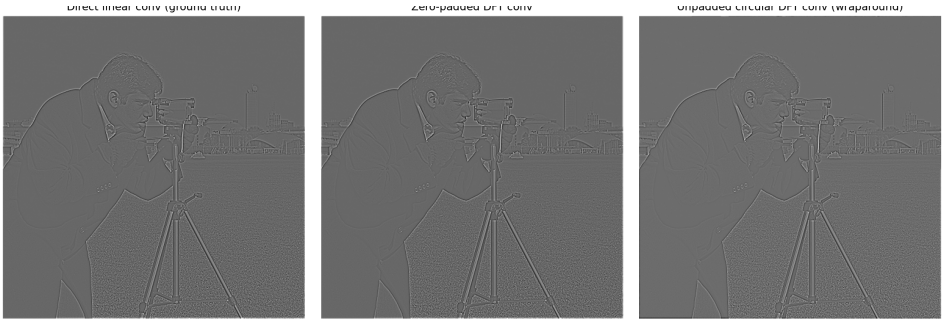


Figure 4: Full-image comparison: direct spatial / zero-padded frequency-domain convolution (left) vs. unpadded circular convolution (right). Wraparound corruption appears as a faint dark/bright seam along the image edges in the circular-convolution result.



Figure 5: Zoomed top rows: the direct/padded result is near-black at the very top edge, while the circular-convolution result shows visibly lighter corruption, since it mixes in content wrapped around from the bottom of the image.

Minimal zero-padding derivation. For an $N \times N$ image and an $M \times M$ kernel, linear convolution is supported on $N + M - 1$ taps per axis (N kernel positions over the image, extended by $M - 1$ by the kernel's own extent). Circular convolution on a $P \times P$ grid equals

this linear convolution exactly only if the grid is large enough to avoid wraparound aliasing, i.e.

$$P \geq N + M - 1 \quad \Longrightarrow \quad N_{x,\text{pad}} = N_{y,\text{pad}} = N + M - 1. \tag{15}$$

For $N = 512$, $M = 5$: $N_{\text{pad}} = 512 + 5 - 1 = 516$, matching the value used for the zero-padded result above (error at float64 precision, confirming the derivation).

Computational trade-off. With $P = 516$ against $N = 512$:

Metric	Value
memory allocation factor, $(P/N)^2$	1.0157
runtime ratio, $(P^2 \log_2 P)/(N^2 \log_2 N)$	1.0170

Table 6: Overhead of zero-padding to the minimal correct size for a small (5×5) kernel.

For a small kernel ($M \ll N$) the overhead is minor ($\sim$1.6–1.7%), but in general the padded array occupies P^2 rather than N^2 floats, and FFT-based convolution costs $O(P^2 \log P)$ rather than $O(N^2 \log N)$. This is precisely the trade-off managed internally by `scipy.signal.fftconvolve`: it does not pad only to the minimal $N + M - 1$, but rounds the padded size up further to a fast ("5-smooth") FFT length, deliberately trading a little extra memory/padding for a potentially large runtime win on top of the $N + M - 1$ correctness floor derived here.

6 Conclusion

A from-scratch, separable-matrix 2-D DFT was implemented and validated against `numpy.fft.fft2` to float64 round-off precision ($\sim 10^{-13}$ relative error). Using this implementation, conjugate symmetry, the Spatial Shift Theorem, and the Circular Convolution Theorem were all empirically confirmed to the same precision, and a synthetic three-sinusoid image's frequencies, amplitudes, and phases were recovered exactly from its spectral peaks, consistent with the derived frequency resolution limit $\Delta f = 1/N$. The phase/magnitude isolation experiments confirmed, both analytically (via the Convolution Theorem, predicted before running any code) and empirically, that spatial structure is carried almost entirely by phase, while magnitude alone (with phase discarded) yields only a symmetric, positionless blob. Finally, the padding experiment demonstrated that unpadded circular convolution via the DFT silently corrupts a border strip of the image through wraparound, that correct zero-padding to $N + M - 1$ eliminates this corruption to numerical precision, and that the resulting memory/runtime overhead is small for small kernels but scales with the same trade-off that production FFT-convolution routines manage via "fast length" rounding. Together these results establish that the DFT, when implemented and used correctly, behaves exactly as its theory predicts: the foundation required for every frequency-domain technique used elsewhere in this assignment.